In [1]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

# Create Spark session
spark = SparkSession.builder.appName("PySpark Extraction Cheat Sheet").master("loca

# Sample DataFrame
data = [
    ("hello world its", ["a", "b", "c"], {"x": 1, "y": 2}, (1, 2), '{"name":"John",
    ("spark sql learning", ["d", "e"], {"z": 9}, (3, 4), '{"name":"Alice"}', "extra
]
df = spark.createDataFrame(data, ["str_col", "arr_col", "map_col", "struct_col", "j

df.show(truncate=False)
```

```
+------------------+---------+---------------+----------+------------------------+-
-------+
|str_col           |arr_col  |map_col        |struct_col|json_col                |m
isc_col|
+------------------+---------+---------------+----------+------------------------+-
-------+
|hello world its   |[a, b, c]|{x -> 1, y -> 2}|{1, 2}    |{"name":"John","age":30}|N
ULL    |
|spark sql learning|[d, e]   |{z -> 9}       |{3, 4}    |{"name":"Alice"}        |e
xtra   |
+------------------+---------+---------------+----------+------------------------+-
-------+
```

1️⃣ Strings

In [2]:
```
# index based slicing in string
df.select(substring(col("str_col"), 1, 5).alias("substring")).show()
```

```
+---------+
|substring|
+---------+
|    hello|
|    spark|
+---------+
```

In [3]:
```
# Substring before/after nth delimiter
df.select(substring_index(col("str_col"), " ", 2).alias("first_word")).show()
```

```
+-----------+
| first_word|
+-----------+
|hello world|
|  spark sql|
+-----------+
```

In [4]:
```
# Regex replace
df.select(regexp_replace(col("str_col"), " ", "_").alias("underscored")).show()
```

```
+------------------+
|       underscored|
+------------------+
|   hello_world_its|
|spark_sql_learning|
+------------------+
```

In [5]:
```
# Split into array
df.select(split(col("str_col"), " ").alias("words")).show(truncate=False)
```

```
+----------------------+
|words                 |
+----------------------+
|[hello, world, its]   |
|[spark, sql, learning]|
+----------------------+
```

In [6]:
```
# Split into array and fetch second word
df.select(split(col("str_col"), " ")[1].alias("words")).show(truncate=False)
```

```
+-----+
|words|
+-----+
|world|
|sql  |
+-----+
```

In [7]:
```
# Padding
df.select(lpad(col("str_col"), 25, "*").alias("lpad")).show(truncate=False)
```

```
+-------------------------+
|lpad                     |
+-------------------------+
|**********hello world its|
|*******spark sql learning|
+-------------------------+
```

In [8]:
```
# Trim
df.select(trim(lit("   padded text   ")).alias("trimmed")).show()
```

```
+-----------+
|    trimmed|
+-----------+
|padded text|
|padded text|
+-----------+
```

In [9]:
```
# Lower / Upper / Initcap
df.select(lower(col("str_col")), upper(col("str_col")), initcap(col("str_col"))).sh
```

```
+------------------+------------------+------------------+
|    lower(str_col)|    upper(str_col)|  initcap(str_col)|
+------------------+------------------+------------------+
|   hello world its|   HELLO WORLD ITS|   Hello World Its|
|spark sql learning|SPARK SQL LEARNING|Spark Sql Learning|
+------------------+------------------+------------------+
```

In [10]:
```
# Reverse string
df.select(reverse(col("str_col")).alias("reversed")).show()
```

```
+------------------+
|          reversed|
+------------------+
|   sti dlrow olleh|
|gninrael lqs kraps|
+------------------+
```

In [11]:
```
# Reverse string words
df.select(concat_ws(" ",reverse(split(col("str_col")," "))).alias("reversed")).show
```

```
+------------------+
|          reversed|
+------------------+
|   its world hello|
|learning sql spark|
+------------------+
```

In [12]:
```
# Character translation
df.select(translate(col("str_col"), "aeiou", "12345").alias("translated")).show()
```

```
+------------------+
|        translated|
+------------------+
|   h2ll4 w4rld 3ts|
|sp1rk sql l21rn3ng|
+------------------+
```

In [13]:
```
# Overlay replace
df.select(overlay(col("str_col"), lit("PY"), 1, 2).alias("overlayed")).show()
```

```
+------------------+
|         overlayed|
+------------------+
|   PYllo world its|
|PYark sql learning|
+------------------+
```

In [14]:
```
# Locate substring
df.select(locate("world", col("str_col")).alias("locate")).show()
```

```
+------+
|locate|
+------+
|     7|
|     0|
+------+
```

2️⃣ Arrays

In [15]:
```
df.show()
```

```
+------------------+---------+---------------+----------+--------------------+-----
---+
|           str_col|  arr_col|         map_col|struct_col|            json_col|misc_
col|
+------------------+---------+---------------+----------+--------------------+-----
---+
|   hello world its|[a, b, c]|{x -> 1, y -> 2}|    {1, 2}|{"name":"John","a...|    N
ULL|
|spark sql learning|   [d, e]|        {z -> 9}|    {3, 4}|    {"name":"Alice"}|   ex
tra|
+------------------+---------+---------------+----------+--------------------+-----
---+
```

In [16]:
```
# 0-based index(first element)
df.select(col("arr_col")[0].alias("first_elem")).show()
```

```
+----------+
|first_elem|
+----------+
|         a|
|         d|
+----------+
```

In [17]:
```
# getItem (same as above)
df.select(col("arr_col").getItem(1).alias("second_elem")).show()
```

```
+-----------+
|second_elem|
+-----------+
|          b|
|          e|
+-----------+
```

In [18]:
```
# element_at (1-based index)
df.select(element_at(col("arr_col"), 2).alias("second_elem")).show()
```

```
+-----------+
|second_elem|
+-----------+
|          b|
|          e|
+-----------+
```

In [19]:
```
# Array contains
df.select(array_contains(col("arr_col"), "b").alias("contains_b")).show()
```

```
+----------+
|contains_b|
+----------+
|      true|
|     false|
+----------+
```

In [20]:
```
# Size of array
df.select(size(col("arr_col")).alias("array_size")).show()
```

```
+----------+
|array_size|
+----------+
|         3|
|         2|
+----------+
```

In [21]:
```
# Sort array
df.select(sort_array(col("arr_col"), asc=False).alias("sorted_desc")).show()
```

```
+-----------+
|sorted_desc|
+-----------+
|  [c, b, a]|
|     [e, d]|
+-----------+
```

In [22]:
```
# Array distinct
df.select(array(lit(1), lit(2), lit(1)).alias("array")).show()
```

```
+---------+
|    array|
+---------+
|[1, 2, 1]|
|[1, 2, 1]|
+---------+
```

In [23]:
```
# Array distinct
df.select(array_distinct(array(lit(1), lit(2), lit(1))).alias("distinct_array")).sh
```

```
+--------------+
|distinct_array|
+--------------+
|        [1, 2]|
|        [1, 2]|
+--------------+
```

In [24]:
```
# Array intersect
df.select(array_intersect(col("arr_col"), array(lit("a"), lit("c"))).alias("interse
```

```
+---------+
|intersect|
+---------+
|   [a, c]|
|       []|
+---------+
```

In [25]:
```
# Array union
df.select(array_union(col("arr_col"), array(lit("z"))).alias("union")).show()
```

```
+------------+
|       union|
+------------+
|[a, b, c, z]|
|   [d, e, z]|
+------------+
```

In [26]:
```
# Array except
df.select(array_except(col("arr_col"), array(lit("a"))).alias("except")).show()
```

```
+------+
|except|
+------+
|[b, c]|
|[d, e]|
+------+
```

In [27]:
```
df.show()
```

```
+------------------+---------+---------------+----------+--------------------+-----
---+
|           str_col|  arr_col|         map_col|struct_col|            json_col|misc_
col|
+------------------+---------+---------------+----------+--------------------+-----
---+
|   hello world its|[a, b, c]|{x -> 1, y -> 2}|    {1, 2}|{"name":"John","a...|    N
ULL|
|spark sql learning|   [d, e]|        {z -> 9}|    {3, 4}|    {"name":"Alice"}|   ex
tra|
+------------------+---------+---------------+----------+--------------------+-----
---+
```

In [28]:
```
# Slice (1-based position)
df.select(slice(col("arr_col"), 1, 2).alias("slice")).show()
```

```
+------+
| slice|
+------+
|[a, b]|
|[d, e]|
+------+
```

In [29]:
```
# Array position
df.select(array_position(col("arr_col"), "b").alias("pos")).show()
```

```
+---+
|pos|
+---+
|  2|
|  0|
+---+
```

In [30]:
```
# Flatten nested arrays
nested_df = spark.createDataFrame([([["a","b"],["c","d"]],)], ["nested_array"])
nested_df.show()
```

```
+----------------+
|    nested_array|
+----------------+
|[[a, b], [c, d]]|
+----------------+
```

In [31]:
```
nested_df.select(flatten(col("nested_array")).alias("flat")).show()
```

```
+------------+
|        flat|
+------------+
|[a, b, c, d]|
+------------+
```

In [32]:
```
# Sequence generation(start, end, step)
spark.range(1).select(sequence(lit(1), lit(5), lit(2)).alias("seq")).show(truncate=
```

```
+---------+
|seq      |
+---------+
|[1, 3, 5]|
+---------+
```

3️⃣ Maps

In [33]:
```
df.show()
```

```
+------------------+---------+---------------+----------+--------------------+-----
---+
|           str_col|  arr_col|         map_col|struct_col|            json_col|misc_
col|
+------------------+---------+---------------+----------+--------------------+-----
---+
|   hello world its|[a, b, c]|{x -> 1, y -> 2}|    {1, 2}|{"name":"John","a...|    N
ULL|
|spark sql learning|   [d, e]|        {z -> 9}|    {3, 4}|    {"name":"Alice"}|   ex
tra|
+------------------+---------+---------------+----------+--------------------+-----
---+
```

In [34]:
```
# Get value by key
df.select(element_at(col("map_col"), lit("x")).alias("value_x")).show()
```

```
+-------+
|value_x|
+-------+
|      1|
|   NULL|
+-------+
```

In [35]:
```
# Keys
df.select(map_keys(col("map_col")).alias("keys")).show(truncate=False)
```

```
+------+
|keys  |
+------+
|[x, y]|
|[z]   |
+------+
```

In [36]:
```
# Values
df.select(map_values(col("map_col")).alias("values")).show(truncate=False)
```

```
+------+
|values|
+------+
|[1, 2]|
|[9]   |
+------+
```

In [37]:
```
# Map entries
df.select(map_entries(col("map_col")).alias("entries")).show(truncate=False)
```

```
+----------------+
|entries         |
+----------------+
|[{x, 1}, {y, 2}]|
|[{z, 9}]        |
+----------------+
```

In [38]:
```
# Map from arrays
spark.createDataFrame([([1, 2], ["a", "b"])], ["keys", "vals"])\
    .select(map_from_arrays(col("keys"), col("vals")).alias("map")).show(truncate=F
```

```
+----------------+
|map             |
+----------------+
|{1 -> a, 2 -> b}|
+----------------+
```

4 Structs

In [39]:
```
df.show()
```

```
+------------------+---------+---------------+----------+--------------------+-----
---+
|           str_col|  arr_col|        map_col|struct_col|            json_col|misc_
col|
+------------------+---------+---------------+----------+--------------------+-----
---+
|   hello world its|[a, b, c]|{x -> 1, y -> 2}|    {1, 2}|{"name":"John","a...|    N
ULL|
|spark sql learning|   [d, e]|       {z -> 9}|    {3, 4}|    {"name":"Alice"}|   ex
tra|
+------------------+---------+---------------+----------+--------------------+-----
---+
```

In [40]:
```
# Access struct field by dot notation
df.select(col("struct_col._1").alias("first_field")).show()
```

```
+-----------+
|first_field|
+-----------+
|          1|
|          3|
+-----------+
```

In [41]:
```
# Access with getField
df.select(col("struct_col").getField("_2").alias("second_field")).show()
```

```
+------------+
|second_field|
+------------+
|           2|
|           4|
+------------+
```

5 JSON

In [42]:
```
df.show(truncate=False)
```

```
+-----------------+---------+---------------+----------+-----------------------+-
-------+
|str_col          |arr_col  |map_col        |struct_col|json_col               |m
isc_col|
+-----------------+---------+---------------+----------+-----------------------+-
-------+
|hello world its  |[a, b, c]|{x -> 1, y -> 2}|{1, 2}    |{"name":"John","age":30}|N
ULL    |
|spark sql learning|[d, e]  |{z -> 9}       |{3, 4}    |{"name":"Alice"}       |e
xtra   |
+-----------------+---------+---------------+----------+-----------------------+-
-------+
```

In [43]:
```
# Extract JSON field
df.select(get_json_object(col("json_col"), "$.name").alias("name")).show()
```

```
+-----+
| name|
+-----+
| John|
|Alice|
+-----+
```

In [44]:
```
# Extract multiple JSON fields
df.select(json_tuple(col("json_col"), "name", "age")).show()
```

```
+-----+----+
|   c0|  c1|
+-----+----+
| John|  30|
|Alice|NULL|
+-----+----+
```

In [45]:
```
# Parse JSON to struct
schema = "name STRING, age INT"
df.select(from_json(col("json_col"), schema).alias("parsed")).show(truncate=False)
```

```
+-------------+
|parsed       |
+-------------+
|{John, 30}   |
|{Alice, NULL}|
+-------------+
```

In [46]:
```
# Struct/array to JSON string
df.select(to_json(struct(col("str_col"), col("arr_col"))).alias("json_string")).sho
```

```
+-----------------------------------------------------+
|json_string                                          |
+-----------------------------------------------------+
|{"str_col":"hello world its","arr_col":["a","b","c"]}|
|{"str_col":"spark sql learning","arr_col":["d","e"]} |
+-----------------------------------------------------+
```

6️⃣ Conditional Extraction

In [47]:
```
df.show(truncate=False)
```

```
+------------------+---------+---------------+----------+------------------------+-
-------+
|str_col           |arr_col  |map_col        |struct_col|json_col                |m
isc_col|
+------------------+---------+---------------+----------+------------------------+-
-------+
|hello world its   |[a, b, c]|{x -> 1, y -> 2}|{1, 2}    |{"name":"John","age":30}|N
ULL    |
|spark sql learning|[d, e]   |{z -> 9}       |{3, 4}    |{"name":"Alice"}        |e
xtra   |
+------------------+---------+---------------+----------+------------------------+-
-------+
```

In [48]:
```
# When / Otherwise
df.select(when(col("misc_col").isNull(), "missing").otherwise("present").alias("sta
```

```
+-------+
| status|
+-------+
|missing|
|present|
+-------+
```

In [49]:
```
# Coalesce
df.select(coalesce(col("misc_col"), lit("default")).alias("coalesced")).show()
```

```
+---------+
|coalesced|
+---------+
|  default|
|    extra|
+---------+
```

There would be many more functions and methods used in PySpark.